CS 300 – Chapter 13 Lectures
Additional Material

Jose de Lima

April 23, 2026

Key idea

Chapter 13 is a grab-bag: three sorting algorithms (one pedagogical, one selection-only, one non-comparison) followed by two linked-list variants (the list-container idiom and the circular list). Each topic is self-contained; there's no unifying narrative. The common thread is "useful ideas that didn't fit anywhere else."

1 13.1 Bubble Sort

The algorithm

Definition —

Bubble sort. Repeatedly walk the array, comparing adjacent pairs and swapping if out of order. After each pass, the largest remaining unsorted element "bubbles" to the end.

```
void bubbleSort(std::vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (a[j] > a[j + 1])
                std::swap(a[j], a[j + 1]);
        }
    }
}
```

The inner loop shrinks each pass because the last i elements are already in their final position.

Cost

- Best case (already sorted): still $O(n^2)$ comparisons in the plain version. An optimization: short-circuit when an entire pass made no swaps, giving $O(n)$ best case.
- Average: $O(n^2)$.
- Worst case (reverse sorted): $O(n^2)$ comparisons, $O(n^2)$ swaps.
- Space: $O(1)$.

- Stable: yes (equal elements never swap past each other).

Why you'll never use bubble sort

Gotcha

Bubble sort is a **teaching tool**, nothing more. For any real data: `std::sort` (introsort), `std::stable_sort` (merge sort), or a radix/bucket sort for integers. Bubble sort's $O(n^2)$ worst case on large inputs is not just slow – it's catastrophically slow. $n = 10^6$ is 10^{12} operations, roughly an hour of CPU time. A real sort does the same job in milliseconds.

Aside

The one defense of bubble sort: its best case with the short-circuit optimization is $O(n)$, matching insertion sort. For nearly-sorted data, it's not terrible. But insertion sort is better on the same workload and easier to reason about, so even that niche has a better answer.

What bubble sort is good for

- Teaching the concept of an in-place comparison sort.
- Interview questions about algorithmic complexity.
- Tiny inputs ($n \leq 20$) where the constant factor is irrelevant.
- Hardware sorting networks, where adjacent-pair compare-and-swap is the primitive operation.

2 13.2 Quickselect

The problem

Definition —

k-th smallest element. Given an array and an integer k, return the k-th smallest value. For $k = 0$, the minimum; for $k = n - 1$, the maximum; for $k = n/2$, the median.

The naive solution: sort, return `a[k]`. That's $O(n \log n)$. Quickselect does it in $O(n)$ average time *without* fully sorting.

The insight

Quickselect borrows Quicksort's partition routine – the one that picks a pivot, puts smaller elements to the left, larger to the right, and returns the pivot's final position.

Key idea

Quicksort recurses on both sides; quickselect recurses on only the side that contains the k-th element. That cuts the total work from $O(n \log n)$ to $O(n)$ on average, because the unused half of each partition is discarded.

Algorithm

```
int partition(std::vector<int>& a, int lo, int hi) {
```

```
    int pivot = a[(lo + hi) / 2];
    int i = lo - 1, j = hi + 1;
    while (true) {
        do { ++i; } while (a[i] < pivot);
        do { --j; } while (a[j] > pivot);
        if (i >= j) return j;
        std::swap(a[i], a[j]);
    }
}

int quickselect(std::vector<int>& a, int lo, int hi, int k) {
    if (lo >= hi) return a[lo];
    int p = partition(a, lo, hi);
    if (k <= p) return quickselect(a, lo, p, k);
    return quickselect(a, p + 1, hi, k);
}
```

Cost

- **Average**: $O(n)$. Partition costs n; the next recursion is on $n/2$; then $n/4$; ... total $n + n/2 + n/4 + \ldots = O(n)$ by geometric sum.
- **Worst**: $O(n^2)$, same as naive quicksort, when pivots are consistently bad.
- **Median-of-medians** gives $O(n)$ worst-case selection, but the constant is large; in practice quickselect with a random or median-of-three pivot is faster.

C++ standard library

You don't have to write quickselect yourself – C++ gives it to you:

```
#include <algorithm>
// Rearranges [begin, end) so nth is the element that would be at
// position n in a fully sorted range. Everything before nth is <=,
// everything after is >=. Average O(n).
std::nth_element(a.begin(), a.begin() + k, a.end());
int kthSmallest = a[k];
```

Key idea

`std::nth_element` is the correct answer to "what's the median / top-k / k-th percentile" in any interview or real codebase. It's linear time, in place, and a single line of code. Reach for it before thinking harder.

Applications

- **Median / percentile computation.**
- **Top-k queries.** After `nth_element`, the first k entries are the top-k (unsorted). If you need them sorted, sort just that prefix.
- **Machine learning**: quickselect-based k-nearest-neighbor scoring.

- **Streaming statistics**: approximate quantiles via repeated quickselect.

3 13.3 Bucket Sort

The idea

Definition —

Bucket sort. Distribute values into K buckets based on a range partition, sort each bucket with another algorithm, concatenate. $O(n)$ average when values are uniformly distributed.

This is a **non-comparison sort** – it exploits numerical structure of the values to beat the $\Omega(n \log n)$ comparison-sort lower bound.

Algorithm

Given non-negative numbers up to some max value M:

1. Create K buckets. Bucket i holds values in $[i \cdot M/K, (i+1) \cdot M/K)$.
2. For each element x, compute `bucket = floor(x * K / (M + 1))` and append x to that bucket.
3. Sort each bucket (insertion sort or any other; each bucket is small).
4. Concatenate buckets in order.

```
void bucketSort(std::vector<int>& a, int K) {
    if (a.empty()) return;
    int maxV = *std::max_element(a.begin(), a.end());
    std::vector<std::vector<int>> buckets(K);

    for (int x : a) {
        int idx = (long long)x * K / (maxV + 1);
        buckets[idx].push_back(x);
    }

    a.clear();
    for (auto& b : buckets) {
        std::sort(b.begin(), b.end());
        a.insert(a.end(), b.begin(), b.end());
    }
}
```

Cost

Let n be the array size and K the bucket count.

- **Distribution pass**: $O(n)$.
- **Per-bucket sort**: if values are uniform, each bucket has $\approx n/K$ elements. Sorting each with $O(m^2)$ insertion sort gives total $K \cdot (n/K)^2 = n^2/K$. With $K = n$, this is $O(n)$.

- **Concatenate**: $O(n + K)$.
- **Total (uniform distribution, $K = n$)**: $O(n)$.
- **Worst case (all elements in one bucket)**: $O(n^2)$.

Gotcha

Bucket sort's $O(n)$ is *contingent on distribution*. If all your values cluster in a narrow range (e.g., 100 random integers between 1 and 10^9), bucket sort degenerates: 99 empty buckets, one bucket with everything. You're paying for K buckets to do nothing. Use bucket sort only when you know the data is roughly uniform over a known range.

Related algorithms

- **Counting sort**: degenerate bucket sort where each bucket holds exactly one value. $O(n + M)$ time, $O(M)$ space. Perfect for small-range integers.
- **Radix sort**: iterate bucket sort on each digit. $O(n \cdot d)$ for d-digit numbers. Used in database sorts, GPU sorts, and networking.
- **Pigeonhole sort**: counting sort by another name, when value range = number of elements.

Key idea

The $n \log n$ barrier only applies to comparison sorts. Any sort that uses numeric / string structure of its inputs – bucket, radix, counting – can beat it for suitable data. Production string sorting (timsort in Python, dpqsort in Java) is comparison-based; production integer sorting (std::sort in libstdc++ at large sizes) can fall through to radix for speed.

4 13.4 List Data Structure

The idiom

A linked list can be represented in two ways:

- **Just a head pointer** (and maybe a tail pointer), held in local variables or as member fields.
- **A wrapper container** – a `List` struct that owns `head`, `tail`, and maybe `size`. Functions take a `List*` instead of a raw head pointer.

```
// Approach 1: just nodes.
struct Node { int data; Node* next; };
Node* head = nullptr;
Node* tail = nullptr;
int size = 0;

// Approach 2: list wrapper.
struct List {
    Node* head = nullptr;
    Node* tail = nullptr;
    int size = 0;
```

```
};

void listPushBack(List* lst, int data) {
    Node* n = new Node{data, nullptr};
    if (!lst->head) lst->head = n;
    else            lst->tail->next = n;
    lst->tail = n;
    ++lst->size;
}
```

Why wrap?

Key idea

The wrapper exists so that list-manipulating functions take *one* parameter instead of three (head, tail, size). More importantly, operations like "insert at tail" or "get length" become $O(1)$ because the struct already knows what the naked-node version would have to recompute.

Operation	Head-only list	Wrapped list
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$ (walk to end)	$O(1)$ (tail ptr)
Get size	$O(n)$ or maintained externally	$O(1)$ (cached)
Pass to function	3 args (head, tail, size)	1 arg (`List*`)

Why not just use `std::list`?

Gotcha

In C++, `std::list` is the wrapper-style doubly-linked list and you should use it unless you have a specific reason not to. Writing your own list is a pedagogical exercise; in production, `std::list` (or `std::forward_list` for singly-linked) is tested, debuggers know how to render it, and you don't leak memory on exception. The "list data structure" the textbook describes is reinventing `std::list`.

When linked lists are the wrong answer

Even with a nice wrapper, linked lists are frequently worse than `std::vector` (dynamic array):

- Cache locality: array elements are contiguous, list nodes scattered across the heap.
- Memory overhead: each node has a `next` pointer, often doubling memory vs. raw values.
- Random access: $O(n)$ for list, $O(1)$ for array.
- Iteration: much slower on a list due to cache misses on every `next`.

Use a linked list when you need cheap insertion/deletion at interior points with existing iterators, or when you're building something that requires pointer stability (e.g., an LRU cache with external references). Otherwise, default to `std::vector`.

5 13.5 Circular Lists

The definition

Definition —

Circular linked list. A linked list where the tail's `next` pointer points back to the head, not `nullptr`. Traversal wraps forever unless explicitly bounded.

```
// Singly-linked circular
struct Node { int data; Node* next; };
// Last node's next points back to head.

// Doubly-linked circular (most common in practice)
struct DNode { int data; DNode* prev; DNode* next; };
// Last node's next -> head; head's prev -> last node.
```

Traversal

The key trick: stop when you come back to the start, not when you hit `nullptr`.

```
void traverseCircular(Node* head) {
    if (!head) return;
    Node* cur = head;
    do {
        visit(cur);
        cur = cur->next;
    } while (cur != head);
}
```

Gotcha

Use do-while, not while. A regular `while (cur != head)` skips the head on the first iteration because we start at head and the condition fails immediately. The do-while runs the body first, then checks the condition.

Why circular lists exist

- **Round-robin scheduling.** OS schedulers cycle through ready processes; each yield pops one and reappends it at the end. A circular list gives O(1) "next process" and O(1) append.
- **Music players on "repeat all."** Queue is a circular list of tracks.
- **Ring buffers** (logically circular, though usually array-based for cache locality).
- **Josephus problem** and other classic cyclic-elimination algorithms – naturally circular.
- **Token-passing networks.** Token Ring Ethernet was literally a circular list of nodes; each station had one successor.
- **Doubly-linked circular lists with a sentinel** give branch-free insert/delete at any position.

Sentinel trick for doubly-linked circular lists

A *sentinel* node – a dummy that's never "real" data but completes the circle – removes the "is this node the head / tail" special cases:

```
struct DNode { int data; DNode* prev; DNode* next; };

struct CircularList {
    DNode sentinel;   // sentinel.next = head, sentinel.prev = tail
    CircularList() { sentinel.prev = sentinel.next = &sentinel; }
};

// Insert n before 'where' -- no null checks needed.
void insertBefore(DNode* where, DNode* n) {
    n->next = where;
    n->prev = where->prev;
    where->prev->next = n;
    where->prev = n;
}

// Remove n -- no null checks needed.
void remove(DNode* n) {
    n->prev->next = n->next;
    n->next->prev = n->prev;
}
```

Key idea

The sentinel technique eliminates all "empty list" and "at the head" edge cases. Insert/remove are six pointer writes with no branching. The Linux kernel's `list_head` uses exactly this structure, and the design has been copied into countless production codebases – it's one of the most elegant tricks in systems programming.

Traversal in reverse (doubly-linked)

Mirror of the forward case – walk `prev` pointers from any starting point until you return:

```
void traverseReverseCircular(DNode* tail) {
    if (!tail) return;
    DNode* cur = tail;
    do {
        visit(cur);
        cur = cur->prev;
    } while (cur != tail);
}
```

Key idea

Chapter takeaway. Bubble sort is useful as a pedagogical example of why most sorting algorithms are $O(n \log n)$. Quickselect solves a narrower problem ("find the k-th element") faster than sorting and is what `std::nth_element` uses internally. Bucket sort shows that when the $\Omega(n \log n)$ comparison

barrier doesn't apply, you can sort in linear time. The list-container idiom is about encapsulating bookkeeping; circular lists are about cyclic problem domains. Nothing in this chapter changes how you think; each piece just adds one more specialized tool to the bag.